File Operations:

```
#include <types.h>
#include <kern/errno.h>
#include <kern/fcntl.h>
#include <kern/stat.h>
#include <lib.h>
#include <proc.h>
#include <current.h>
#include <addrspace.h>
#include <vnode.h>
#include <vfs.h>
#include <uio.h>
#include <copyinout.h>
#include <file_syscalls.h>
#include <limits.h>

int sys_open(const_userptr_t filename, int flags, mode_t mode, int
*retval) {
    char path[PATH_MAX];
    size_t got;
    int result, fd;
    struct openfile *file;

    /* 1. Safely copy the filename from user space to kernel space */
    result = copyinstr(filename, path, PATH_MAX, &got);
    if (result) {
        return result;
    }

    /* 2. Allocate and initialize the openfile struct */
    file = kmalloc(sizeof(struct openfile));
    if (file == NULL) {
        return ENOMEM;
    }
    file->of_offset = 0;
    file->of_accmode = flags & O_ACCMODE;
    file->of_flags = flags;
    file->of_refcount = 1;
    file->of_lock = lock_create("openfile_lock");
```

```
    if (file->of_lock == NULL) {
        kfree(file);
        return ENOMEM;
    }

    /* 3. Open the file via Virtual File System (VFS) */
    result = vfs_open(path, flags, mode, &file->of_vnode);
    if (result) {
        lock_destroy(file->of_lock);
        kfree(file);
        return result;
    }

    /* 4. Find an empty slot in the process's file descriptor table */
    for (fd = 0; fd < OPEN_MAX; fd++) {
        if (curproc->p_fdtable[fd] == NULL) {
            curproc->p_fdtable[fd] = file;
            *retval = fd;
            return 0;
        }
    }

    /* If we get here, the FD table is full */
    vfs_close(file->of_vnode);
    lock_destroy(file->of_lock);
    kfree(file);
    return EMFILE;
}

int sys_read(int fd, userptr_t buf, size_t buflen, int *retval) {
    struct openfile *file;
    struct iovec iov;
    struct uio u;
    int result;

    bzero(&iov, sizeof(iov));
    bzero(&u, sizeof(u));

    /* Validate FD */
    if (fd < 0 || fd >= OPEN_MAX || curproc->p_fdtable[fd] == NULL) {
```

```
        return EBADF;
    }
    file = curproc->p_fdtable[fd];

    /* Check access mode */
    if (file->of_accmode == O_WRONLY) {
        return EBADF;
    }

    lock_acquire(file->of_lock);

    /* Setup UIO for reading to user space */
    iov.iov_ubase = buf;
    iov.iov_len = buflen;
    u.uio_iov = &iov;
    u.uio_iovcnt = 1;
    u.uio_offset = file->of_offset;
    u.uio_resid = buflen;
    u.uio_segflg = UIO_USERSPACE;
    u.uio_rw = UIO_READ;
    u.uio_space = curproc->p_addrspace;

    result = VOP_READ(file->of_vnode, &u);
    if (result) {
        lock_release(file->of_lock);
        return result;
    }

    /* Update offset and return bytes read */
    file->of_offset = u.uio_offset;
    *retval = buflen - u.uio_resid;

    lock_release(file->of_lock);
    return 0;
}

int sys_write(int fd, const_userptr_t buf, size_t nbytes, int *retval) {
    struct openfile *file;
    struct iovec iov;
    struct uio u;
```

```
    int result;

    bzero(&iov, sizeof(iov));
    bzero(&u, sizeof(u));

    /* Validate FD */
    if (fd < 0 || fd >= OPEN_MAX || curproc->p_fdtable[fd] == NULL) {
        return EBADF;
    }
    file = curproc->p_fdtable[fd];

    /* Check access mode */
    if (file->of_accmode == O_RDONLY) {
        return EBADF;
    }

    lock_acquire(file->of_lock);

    /* APPEND LOGIC: If file was opened with O_APPEND, force offset to end
of file */
    if (file->of_flags & O_APPEND) {
        struct stat statbuf;
        result = VOP_STAT(file->of_vnode, &statbuf);
        if (result == 0) {
            file->of_offset = statbuf.st_size;
        }
    }

    /* Setup UIO for writing from user space */
    iov.iov_ubase = (userptr_t)buf; /* Cast away const to satisfy iovec
struct */
    iov.iov_len = nbytes;
    u.uio_iov = &iov;
    u.uio_iovcnt = 1;
    u.uio_offset = file->of_offset;
    u.uio_resid = nbytes;
    u.uio_segflg = UIO_USERSPACE;
    u.uio_rw = UIO_WRITE;
    u.uio_space = curproc->p_addrspace;
```

```
    result = VOP_WRITE(file->of_vnode, &u);
    if (result) {
        lock_release(file->of_lock);
        return result;
    }

    /* Update offset and return bytes written */
    file->of_offset = u.uio_offset;
    *retval = nbytes - u.uio_resid;

    lock_release(file->of_lock);
    return 0;
}

int sys_close(int fd) {
    struct openfile *file;

    /* Validate FD */
    if (fd < 0 || fd >= OPEN_MAX || curproc->p_fdtable[fd] == NULL) {
        return EBADF;
    }

    file = curproc->p_fdtable[fd];

    /* Remove from current process FD table */
    curproc->p_fdtable[fd] = NULL;

    lock_acquire(file->of_lock);
    file->of_refcount--;

    if (file->of_refcount == 0) {
        vfs_close(file->of_vnode);
        lock_release(file->of_lock);
        lock_destroy(file->of_lock);
        kfree(file);
    } else {
        lock_release(file->of_lock);
    }

    return 0;
```

```
}

int sys_remove(const_userptr_t filename) {
    char path[PATH_MAX];
    size_t got;

    return copyinstr(filename, path, PATH_MAX, &got) ? : vfs_remove(path);
}

int sys_fstat(int fd, userptr_t statbuf) {
    struct openfile *file;
    struct stat kstat;
    int result;

    if (fd < 0 || fd >= OPEN_MAX || curproc->p_fdtable[fd] == NULL) {
        return EBADF;
    }

    file = curproc->p_fdtable[fd];

    lock_acquire(file->of_lock);
    result = VOP_STAT(file->of_vnode, &kstat);
    lock_release(file->of_lock);
    if (result) {
        return result;
    }

    return copyout(&kstat, statbuf, sizeof(kstat));
}

int sys_getdirentry(int fd, userptr_t buf, size_t buflen, int *retval) {
    struct openfile *file;
    struct iovec iov;
    struct uio u;
    int result;

    if (fd < 0 || fd >= OPEN_MAX || curproc->p_fdtable[fd] == NULL) {
        return EBADF;
    }
```

```
    file = curproc->p_fdtable[fd];

    lock_acquire(file->of_lock);

    iov.iov_ubase = buf;
    iov.iov_len = buflen;
    u.uio_iov = &iov;
    u.uio_iovcnt = 1;
    u.uio_offset = file->of_offset;
    u.uio_resid = buflen;
    u.uio_segflg = UIO_USERSPACE;
    u.uio_rw = UIO_READ;
    u.uio_space = curproc->p_addrspace;

    result = VOP_GETDIRENTRY(file->of_vnode, &u);
    if (result) {
        lock_release(file->of_lock);
        return result;
    }

    file->of_offset = u.uio_offset;
    *retval = buflen - u.uio_resid;

    lock_release(file->of_lock);
    return 0;
}

int stdio_init(void) {
    char con1[] = "con:";
    char con2[] = "con:";
    char con3[] = "con:";
    struct vnode *v0, *v1, *v2;
    int result;

    /* Initialize fd 0 (stdin) */
    result = vfs_open(con1, O_RDONLY, 0, &v0);
    if (result) return result;

    curproc->p_fdtable[0] = kmalloc(sizeof(struct openfile));
    curproc->p_fdtable[0]->of_vnode = v0;
```

```
    curproc->p_fdtable[0]->of_offset = 0;
    curproc->p_fdtable[0]->of_accmode = O_RDONLY;
    curproc->p_fdtable[0]->of_flags = O_RDONLY;
    curproc->p_fdtable[0]->of_refcount = 1;
    curproc->p_fdtable[0]->of_lock = lock_create("fd0_lock");

    /* Initialize fd 1 (stdout) */
    result = vfs_open(con2, O_WRONLY, 0, &v1);
    if (result) return result;

    curproc->p_fdtable[1] = kmalloc(sizeof(struct openfile));
    curproc->p_fdtable[1]->of_vnode = v1;
    curproc->p_fdtable[1]->of_offset = 0;
    curproc->p_fdtable[1]->of_accmode = O_WRONLY;
    curproc->p_fdtable[1]->of_flags = O_WRONLY;
    curproc->p_fdtable[1]->of_refcount = 1;
    curproc->p_fdtable[1]->of_lock = lock_create("fd1_lock");

    /* Initialize fd 2 (stderr) */
    result = vfs_open(con3, O_WRONLY, 0, &v2);
    if (result) return result;

    curproc->p_fdtable[2] = kmalloc(sizeof(struct openfile));
    curproc->p_fdtable[2]->of_vnode = v2;
    curproc->p_fdtable[2]->of_offset = 0;
    curproc->p_fdtable[2]->of_accmode = O_WRONLY;
    curproc->p_fdtable[2]->of_flags = O_WRONLY;
    curproc->p_fdtable[2]->of_refcount = 1;
    curproc->p_fdtable[2]->of_lock = lock_create("fd2_lock");

    return 0;
}
```

This is the file where i have implemented fileoperations for my os161. The output is showing like:

os161user@2a20984d46d8:~/os161/root$ sys161 kernel "mount sfs lhd0;p /testbin/filetest file.txt;q"
sys161: System/161 release 2.0.8, compiled Mar 22 2022 23:45:01

OS/161 base system version 2.0.3

```
Copyright (c) 2000, 2001-2005, 2008-2011, 2013, 2014
   President and Fellows of Harvard College.  All rights reserved.

Put-your-group-name-here's system version 0 (DUMBVM #31)

348k physical memory available
Device probe...
lamebus0 (system main bus)
emu0 at lamebus0
ltrace0 at lamebus0
ltimer0 at lamebus0
beep0 at ltimer0
rtclock0 at ltimer0
lrandom0 at lamebus0
random0 at lrandom0
lhd0 at lamebus0
lhd1 at lamebus0
lser0 at lamebus0
con0 at lser0

cpu0: MIPS/161 (System/161 2.x) features 0x0
OS/161 kernel: mount sfs lhd0
vfs: Mounted os161disk: on lhd0
Operation took 0.139519610 seconds
OS/161 kernel: p /testbin/filetest file.txt
Passed filetest.
Operation took 18.055931920 seconds
OS/161 kernel: q
Shutting down.
vfs: Unmounting lhd0:
The system is halted.
sys161: 525415310 cycles (489526454 run, 35888856 global-idle)
sys161:   cpu0: 455325387 kern, 651 user, 0 idle; 11602 ll, 11602/0 sc, 81530 sync
sys161: 2630 irqs 48 exns 4r/0w disk 0r/805w console 10r/2w/7m emufs 0r/0w net
sys161: Elapsed real time: 3.309226 seconds (158.773 mhz)
sys161: Elapsed virtual time: 19.652885233 seconds (25 mhz)
os161user@2a20984d46d8:~/os161/root$
```

The following code triggers as a test code for getting the output results.

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <err.h>
```

```
int
main(int argc, char *argv[])
{
    static char writebuf[41] = "Twiddle dee dee, Twiddle dum
dum.......\n";
    static char readbuf[41];

    const char *file;
    int fd, rv;

    if (argc == 0) {
        warnx("No arguments - running on \"testfile\"");
        file = "testfile";
    }
    else if (argc == 2) {
        file = argv[1];
    }
    else {
        errx(1, "Usage: filetest <filename>");
    }

    fd = open(file, O_WRONLY|O_CREAT|O_TRUNC, 0664);
    if (fd<0) {
        err(1, "%s: open for write", file);
    }



    rv = write(fd, writebuf, 40);
    if (rv<0) {
        err(1, "%s: write", file);
    }

    rv = close(fd);
    if (rv<0) {
        err(1, "%s: close (1st time)", file);
    }

    fd = open(file, O_RDONLY);
    if (fd<0) {
```

```
        err(1, "%s: open for read", file);
    }

    rv = read(fd, readbuf, 40);
    if (rv<0) {
        err(1, "%s: read", file);
    }
    rv = close(fd);
    if (rv<0) {
        err(1, "%s: close (2nd time)", file);
    }
    /* ensure null termination */
    readbuf[40] = 0;

    if (strcmp(readbuf, writebuf)) {
        errx(1, "Buffer data mismatch!");
    }

    printf("Passed filetest.\n");
    return 0;
}
```

System Calls:

```
        case SYS__exit:
            sys__exit(tf->tf_a0);
            panic("sys__exit returned");

         case SYS_fork:
            err = sys_fork(tf, &retval);
            break;

         case SYS_getpid:
            err = sys_getpid((pid_t *)&retval);
            break;

        case SYS_waitpid:
             err = sys_waitpid((pid_t)tf->tf_a0,
```

```
                (userptr_t)tf->tf_a1,
                (int)tf->tf_a2,
                (pid_t *)&retval);
            break;

        case SYS_execv:
            err = sys_execv((const_userptr_t)tf->tf_a0,
                (userptr_t *)tf->tf_a1);
            break;

            default:
                kprintf("Unknown syscall %d\n", callno);
                err = ENOSYS;
                break;
    }
```

• fork
• getpid
• waitpid
• exit
• execv

Are implemented here in the codebase.

```
#include <types.h>
#include <kern/errno.h>
#include <lib.h>
#include <proc.h>
#include <current.h>
#include <syscall.h>
#include <thread.h>
#include <addrspace.h>
#include <mips/trapframe.h>
#include <file_syscalls.h>
#include <kern/wait.h>
#include <kern/fcntl.h>
#include <copyinout.h>
#include <limits.h>
#include <vm.h>
#include <vfs.h>
#include <test.h>

#define MAX_PROC 100
```

```
extern struct proc *process_table[MAX_PROC];

static void
fork_copy_fdtables(struct proc *src, struct proc *dst)
{
    for (int fd = 0; fd < OPEN_MAX; fd++) {
        struct openfile *file = src->p_fdtable[fd];

        if (file == NULL) {
            dst->p_fdtable[fd] = NULL;
            continue;
        }

        lock_acquire(file->of_lock);
        file->of_refcount++;
        lock_release(file->of_lock);
        dst->p_fdtable[fd] = file;
    }
}

int
sys_getpid(pid_t *retval)
{
    if (curproc == NULL) {
        return ESRCH;
    }

    *retval = curproc->p_pid;

    return 0;
}

int
sys_fork(struct trapframe *tf, pid_t *retval)
{
    struct proc *newproc;
    struct addrspace *newaddr;
    struct trapframe *newtf;
    int result;
```

```
    newproc = proc_create_runprogram("child");
    if (newproc == NULL) return ENOMEM;

    result = as_copy(curproc->p_addrspace, &newaddr);
    if (result) {
        proc_destroy(newproc);
        return result;
    }
    newproc->p_addrspace = newaddr;

    newtf = kmalloc(sizeof(struct trapframe));
    if (newtf == NULL) {
        proc_destroy(newproc);
        return ENOMEM;
    }
    *newtf = *tf;

    newproc->p_ppid = curproc->p_pid;

    fork_copy_fdtables(curproc, newproc);

    result = thread_fork("child", newproc, enter_forked_process, newtf,
0);
    if (result) {
        proc_destroy(newproc);
        return result;
    }

    *retval = newproc->p_pid;
    return 0;
}

int
sys_waitpid(pid_t pid, userptr_t status, int options, pid_t *retval)
{
    struct proc *child;
    int exit_status;
    int result;

    (void)options;
```

```
    if (pid <= 0 || pid >= MAX_PROC) {
        return EINVAL;
    }

    if (status == NULL) {
        return EFAULT;
    }

    spinlock_acquire(&curproc->p_lock);
    child = process_table[pid];
    spinlock_release(&curproc->p_lock);

    if (child == NULL) {
        return ECHILD;
    }

    if (child->p_ppid != curproc->p_pid) {
        return ECHILD;
    }

    if (!child->p_exited) {
        P(child->p_exitsem);
    }

    exit_status = child->p_status;

    result = copyout(&exit_status, status, sizeof(int));
    if (result) {
        return result;
    }

    *retval = pid;
    proc_destroy(child);

    return 0;
}

void
sys__exit(int exitcode)
```

```
{
    int status;

    status = _MKWAIT_EXIT(exitcode);

    curproc->p_status = status;
    curproc->p_exited = true;

    if (curproc->p_ppid > 0) {
        struct proc *parent;
        parent = process_table[curproc->p_ppid];
        if (parent != NULL && parent->p_exitsem != NULL) {
            V(curproc->p_exitsem);
        }
    }

    thread_exit();
    panic("thread_exit returned\n");
}

static int
copy_argv_to_stack(int argc, char **argv, vaddr_t stackptr,
           userptr_t *user_argv_out)
{
    int i, result;
    size_t len;
    vaddr_t sp;
    vaddr_t *str_addrs;

    size_t strings_size = 0;
    for (i = 0; i < argc; i++) {
        len = strlen(argv[i]) + 1;
        len = (len + 3) & ~3;
        strings_size += len;
    }

    size_t ptrs_size = (argc + 1) * sizeof(char *);

    sp = stackptr - strings_size - ptrs_size;
    sp &= ~3;
```

```
vaddr_t strings_base = sp + ptrs_size;

str_addrs = kmalloc(sizeof(vaddr_t) * argc);
if (str_addrs == NULL) {
    return ENOMEM;
}

vaddr_t str_ptr = strings_base;
for (i = 0; i < argc; i++) {
    len = strlen(argv[i]) + 1;
    str_addrs[i] = str_ptr;
    result = copyoutstr(argv[i], (userptr_t)str_ptr, len, NULL);
    if (result) {
        kfree(str_addrs);
        return result;
    }
    str_ptr += (len + 3) & ~3;
}

vaddr_t ptr = sp;
for (i = 0; i < argc; i++) {
    result = copyout(&str_addrs[i], (userptr_t)ptr, sizeof(vaddr_t));
    if (result) {
        kfree(str_addrs);
        return result;
    }
    ptr += sizeof(vaddr_t);
}

vaddr_t null_ptr = 0;
result = copyout(&null_ptr, (userptr_t)ptr, sizeof(vaddr_t));
if (result) {
    kfree(str_addrs);
    return result;
}

kfree(str_addrs);
*user_argv_out = (userptr_t)sp;
return 0;
```

```
}

int
sys_execv(const_userptr_t path, userptr_t *argv)
{
    char kpath[PATH_MAX];
    size_t got;
    int result;
    struct vnode *v;
    vaddr_t entrypoint, stackptr;
    struct addrspace *as, *oldas;
    int kargc;
    char **kargv;
    char *kargstrings[64];
    userptr_t user_argv;

    result = copyinstr(path, kpath, PATH_MAX, &got);
    if (result) {
        return result;
    }

    kargc = 0;
    if (argv != NULL) {
        userptr_t *user_argv_array = argv;
        for (kargc = 0; kargc < 63; kargc++) {
            userptr_t uarg;
            result = copyin((const_userptr_t)(user_argv_array + kargc),
&uarg, sizeof(userptr_t));
            if (result) {
                return result;
            }
            if (uarg == NULL) {
                break;
            }
            kargstrings[kargc] = kmalloc(PATH_MAX);
            if (kargstrings[kargc] == NULL) {
                result = ENOMEM;
                goto fail_cleanup;
            }
            result = copyinstr(uarg, kargstrings[kargc], PATH_MAX, NULL);
```

```
            if (result) {
                goto fail_cleanup;
            }
        }
    } else {
        kargstrings[0] = kmalloc(strlen(kpath) + 1);
        if (kargstrings[0] == NULL) {
            return ENOMEM;
        }
        strcpy(kargstrings[0], kpath);
        kargc = 1;
    }

    kargv = kmalloc(sizeof(char *) * (kargc + 1));
    if (kargv == NULL) {
        result = ENOMEM;
        goto fail_cleanup;
    }
    for (int i = 0; i < kargc; i++) {
        kargv[i] = kargstrings[i];
    }
    kargv[kargc] = NULL;

    result = vfs_open(kpath, O_RDONLY, 0, &v);
    if (result) {
        goto fail_cleanup2;
    }

    /*
     * Save the old address space. We create the new one and load the
     * ELF into it. Only destroy the old AS after we know the new one
works.
     * If anything fails, we restore the old AS so the process can
     * return to user mode and continue.
     */
    oldas = curproc->p_addrspace;

    as = as_create();
    if (as == NULL) {
        vfs_close(v);
```

```
        result = ENOMEM;
        if (oldas != NULL) {
            proc_setas(oldas);
        }
        goto fail_cleanup2;
    }
    proc_setas(as);
    as_activate();

    result = load_elf(v, &entrypoint);
    vfs_close(v);
    if (result) {
        /* Restore old address space on failure */
        as_destroy(as);
        proc_setas(oldas);
        if (oldas != NULL) {
            as_activate();
        }
        goto fail_cleanup2;
    }

    result = as_define_stack(as, &stackptr);
    if (result) {
        as_destroy(as);
        proc_setas(oldas);
        if (oldas != NULL) {
            as_activate();
        }
        goto fail_cleanup2;
    }

    user_argv = NULL;
    if (kargc > 0) {
        result = copy_argv_to_stack(kargc, kargv, stackptr, &user_argv);
        if (result) {
            as_destroy(as);
            proc_setas(oldas);
            if (oldas != NULL) {
                as_activate();
            }
```

```
            goto fail_cleanup2;
        }
    }

    /*
     * Everything succeeded. Now destroy the old address space.
     */
    if (oldas != NULL) {
        as_destroy(oldas);
    }

    for (int i = 0; i < kargc; i++) {
        kfree(kargv[i]);
    }
    kfree(kargv);

    enter_new_process(kargc, user_argv, NULL, stackptr, entrypoint);
    panic("enter_new_process returned\n");
    return EINVAL;

fail_cleanup2:
    kfree(kargv);
fail_cleanup:
    for (int i = 0; i < kargc; i++) {
        if (kargstrings[i] != NULL) {
            kfree(kargstrings[i]);
        }
    }
    return result;
}

void
enter_forked_process(void *data1, unsigned long data2)
{
    struct trapframe *tf_passed = (struct trapframe *)data1;
    struct trapframe tf = *tf_passed;

    kfree(tf_passed);
    (void)data2;
```

```
    tf.tf_v0 = 0;
    tf.tf_a3 = 0;
    tf.tf_epc += 4;

    as_activate();

    mips_usermode(&tf);
}
```

```
os161user@2a20984d46d8:~/os161/root$ sys161 -Z 20 kernel "mount sfs lhd0;p
/testbin/proctest;q"
sys161: System/161 release 2.0.8, compiled Mar 22 2022 23:45:01

OS/161 base system version 2.0.3
Copyright (c) 2000, 2001-2005, 2008-2011, 2013, 2014
   President and Fellows of Harvard College.  All rights reserved.

Put-your-group-name-here's system version 0 (DUMBVM #31)

348k physical memory available
Device probe...
lamebus0 (system main bus)
emu0 at lamebus0
ltrace0 at lamebus0
ltimer0 at lamebus0
beep0 at ltimer0
rtclock0 at ltimer0
lrandom0 at lamebus0
random0 at lrandom0
lhd0 at lamebus0
lhd1 at lamebus0
lser0 at lamebus0
con0 at lser0

cpu0: MIPS/161 (System/161 2.x) features 0x0
OS/161 kernel: mount sfs lhd0
vfs: Mounted os161disk: on lhd0
Operation took 0.139241911 seconds
OS/161 kernel: p /testbin/proctest
=== Process System Call Tests ===
```

```
Test getpid:
  getpid() returned 2
  [PASS] getpid
Test fork+waitpid+exit:
  Child: PID is 3 (parent PID is 2)
  Parent: child 3 exited with status 42
  [PASS] fork+waitpid+exit
Test execv:
The current process PID is: 3
  Child execv'd mygetpid successfully
  [PASS] execv

=== Results: 3/3 passed ===
Operation took 18.189296440 seconds
OS/161 kernel: q
Shutting down.
vfs: Unmounting lhd0:
The system is halted.
sys161: 527401203 cycles (492175797 run, 35225406 global-idle)
sys161:   cpu0: 458616239 kern, 15650 user, 0 idle; 18268 ll, 18268/0 sc, 96868 sync
sys161: 2972 irqs 963 exns 4r/0w disk 0r/1130w console 18r/0w/5m emufs 0r/0w net
sys161: Elapsed real time: 3.829501 seconds (137.721 mhz)
sys161: Elapsed virtual time: 19.758581254 seconds (25 mhz)
os161user@2a20984d46d8:~/os161/root$
```

```c
/*
 * proctest.c
 *
 * Tests the process-related system calls: fork, getpid, waitpid, exit,
execv.
 * Uses at most 2 concurrent forks to stay within dumbvm's 348K memory
limit.
 */

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>

static int tests_run = 0;
static int tests_passed = 0;
```

```c
static void test_pass(const char *name) {
    tests_run++;
    tests_passed++;
    printf("  [PASS] %s\n", name);
}

static void test_fail(const char *name, const char *reason) {
    tests_run++;
    printf("  [FAIL] %s: %s\n", name, reason);
}

/*
 * Test 1: getpid returns a valid PID
 */
static void test_getpid(void) {
    int pid;
    printf("Test getpid:\n");
    pid = getpid();
    printf("  getpid() returned %d\n", pid);
    if (pid > 0) {
        test_pass("getpid");
    } else {
        test_fail("getpid", "PID <= 0");
    }
}

/*
 * Test 2: fork + waitpid + exit + getpid correctness
 * Fork a child that verifies its PID differs from parent, exits with 42.
 * Parent waits and verifies the exit status.
 */
static void test_fork_wait_exit(void) {
    int pid, status;
    int parent_pid = getpid();
    printf("Test fork+waitpid+exit:\n");
    pid = fork();
    if (pid < 0) {
        test_fail("fork+waitpid+exit", "fork returned negative");
        return;
```

```
    }
    if (pid == 0) {
        /* Child: verify PID is different, exit with code 42 */
        int child_pid = getpid();
        if (child_pid == parent_pid) {
            printf("  FAIL: child PID %d == parent PID %d\n", child_pid,
parent_pid);
            exit(1);
        }
        printf("  Child: PID is %d (parent PID is %d)\n", child_pid,
parent_pid);
        exit(42);
    }
    /* Parent: wait and check status */
    if (waitpid(pid, &status, 0) < 0) {
        test_fail("fork+waitpid+exit", "waitpid failed");
        return;
    }
    if (WIFEXITED(status) && WEXITSTATUS(status) == 42) {
        printf("  Parent: child %d exited with status 42\n", pid);
        test_pass("fork+waitpid+exit");
    } else {
        test_fail("fork+waitpid+exit", "unexpected exit status");
    }
}

/*
 * Test 3: execv replaces process image
 * Fork a child, have it exec mygetpid, parent checks exit status.
 */
static void test_execv(void) {
    int pid, status;
    char path[] = "/testbin/mygetpid";
    char arg0[] = "mygetpid";
    char *child_argv[2];

    child_argv[0] = arg0;
    child_argv[1] = NULL;

    printf("Test execv:\n");
```

```
    pid = fork();
    if (pid < 0) {
        test_fail("execv", "fork failed");
        return;
    }
    if (pid == 0) {
        execv(path, child_argv);
        /* If execv fails */
        exit(1);
    }
    waitpid(pid, &status, 0);
    if (WIFEXITED(status) && WEXITSTATUS(status) == 0) {
        printf("  Child execv'd mygetpid successfully\n");
        test_pass("execv");
    } else {
        test_fail("execv", "child execv failed");
    }
}

int main(int argc, char *argv[]) {
    (void)argc;
    (void)argv;

    printf("=== Process System Call Tests ===\n\n");

    test_getpid();
    test_fork_wait_exit();
    test_execv();

    printf("\n=== Results: %d/%d passed ===\n", tests_passed, tests_run);
    return (tests_passed == tests_run) ? 0 : 1;
}
```

This is the test code for the system call test.

For lock and CV:

```
/*
 * Copyright (c) 2000, 2001, 2002, 2003, 2004, 2005, 2008, 2009
```

```
 *      The President and Fellows of Harvard College.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 * 3. Neither the name of the University nor the names of its contributors
 *    may be used to endorse or promote products derived from this
software
 *    without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE UNIVERSITY AND CONTRIBUTORS ``AS IS''
AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE
 * ARE DISCLAIMED.  IN NO EVENT SHALL THE UNIVERSITY OR CONTRIBUTORS BE
LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY
WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 */

/*
 * Synchronization primitives.
 * The specifications of the functions are in synch.h.
 */

#include <types.h>
```

```
#include <lib.h>
#include <spinlock.h>
#include <wchan.h>
#include <thread.h>
#include <current.h>
#include <synch.h>

////////////////////////////////////////////////////////////
//
// Semaphore.

struct semaphore *
sem_create(const char *name, unsigned initial_count)
{
        struct semaphore *sem;

        sem = kmalloc(sizeof(*sem));
        if (sem == NULL) {
                return NULL;
        }

        sem->sem_name = kstrdup(name);
        if (sem->sem_name == NULL) {
                kfree(sem);
                return NULL;
        }

        sem->sem_wchan = wchan_create(sem->sem_name);
        if (sem->sem_wchan == NULL) {
                kfree(sem->sem_name);
                kfree(sem);
                return NULL;
        }

        spinlock_init(&sem->sem_lock);
        sem->sem_count = initial_count;

        return sem;
}
```

```
void
sem_destroy(struct semaphore *sem)
{
        KASSERT(sem != NULL);

        /* wchan_cleanup will assert if anyone's waiting on it */
        spinlock_cleanup(&sem->sem_lock);
        wchan_destroy(sem->sem_wchan);
        kfree(sem->sem_name);
        kfree(sem);
}

void
P(struct semaphore *sem)
{
        KASSERT(sem != NULL);

        /*
         * May not block in an interrupt handler.
         *
         * For robustness, always check, even if we can actually
         * complete the P without blocking.
         */
        KASSERT(curthread->t_in_interrupt == false);

        /* Use the semaphore spinlock to protect the wchan as well. */
        spinlock_acquire(&sem->sem_lock);
        while (sem->sem_count == 0) {
                /*
                 *
                 * Note that we don't maintain strict FIFO ordering of
                 * threads going through the semaphore; that is, we
                 * might "get" it on the first try even if other
                 * threads are waiting. Apparently according to some
                 * textbooks semaphores must for some reason have
                 * strict ordering. Too bad. :-)
                 *
                 * Exercise: how would you implement strict FIFO
                 * ordering?
                 */
```

```
                wchan_sleep(sem->sem_wchan, &sem->sem_lock);
        }
        KASSERT(sem->sem_count > 0);
        sem->sem_count--;
        spinlock_release(&sem->sem_lock);
}

void
V(struct semaphore *sem)
{
        KASSERT(sem != NULL);

        spinlock_acquire(&sem->sem_lock);

        sem->sem_count++;
        KASSERT(sem->sem_count > 0);
        wchan_wakeone(sem->sem_wchan, &sem->sem_lock);

        spinlock_release(&sem->sem_lock);
}

////////////////////////////////////////////////////////////
//
// Lock.

struct lock *
lock_create(const char *name)
{
        struct lock *lock;

        lock = kmalloc(sizeof(*lock));
        if (lock == NULL) {
                return NULL;
        }

        lock->lk_name = kstrdup(name);
        if (lock->lk_name == NULL) {
                kfree(lock);
                return NULL;
        }
```

```
        HANGMAN_LOCKABLEINIT(&lock->lk_hangman, lock->lk_name);

        // Initialize wait channel
        lock->lk_wchan = wchan_create(lock->lk_name);
        if (lock->lk_wchan == NULL) {
                kfree(lock->lk_name);
                kfree(lock);
                return NULL;
        }

        // Initialize spinlock and owner
        spinlock_init(&lock->lk_spinlock);
        lock->lk_owner = NULL;

        return lock;
}

void
lock_destroy(struct lock *lock)
{
        KASSERT(lock != NULL);

        // Ensure the lock isn't held by anyone when being destroyed
        KASSERT(lock->lk_owner == NULL);

        spinlock_cleanup(&lock->lk_spinlock);
        wchan_destroy(lock->lk_wchan);

        kfree(lock->lk_name);
        kfree(lock);
}

void
lock_acquire(struct lock *lock)
{
        KASSERT(lock != NULL);
        KASSERT(curthread->t_in_interrupt == false); // May not block in
an interrupt
```

```
        KASSERT(!lock_do_i_hold(lock));              // Prevent recursive
deadlocks

        spinlock_acquire(&lock->lk_spinlock);

        /* Call this (atomically) before waiting for a lock */
        HANGMAN_WAIT(&curthread->t_hangman, &lock->lk_hangman);

        while (lock->lk_owner != NULL) {
                wchan_sleep(lock->lk_wchan, &lock->lk_spinlock);
        }

        lock->lk_owner = curthread;

        /* Call this (atomically) once the lock is acquired */
        HANGMAN_ACQUIRE(&curthread->t_hangman, &lock->lk_hangman);

        spinlock_release(&lock->lk_spinlock);
}

void
lock_release(struct lock *lock)
{
        KASSERT(lock != NULL);
        KASSERT(lock_do_i_hold(lock)); // Only the lock owner can release
it

        spinlock_acquire(&lock->lk_spinlock);

        /* Call this (atomically) when the lock is released */
        HANGMAN_RELEASE(&curthread->t_hangman, &lock->lk_hangman);

        lock->lk_owner = NULL;
        wchan_wakeone(lock->lk_wchan, &lock->lk_spinlock);

        spinlock_release(&lock->lk_spinlock);
}

bool
lock_do_i_hold(struct lock *lock)
```

```
{
        bool do_i_hold;

        KASSERT(lock != NULL);

        // Protect the check with the spinlock
        spinlock_acquire(&lock->lk_spinlock);
        do_i_hold = (lock->lk_owner == curthread);
        spinlock_release(&lock->lk_spinlock);

        return do_i_hold;
}

////////////////////////////////////////////////////////////
//
// CV


struct cv *
cv_create(const char *name)
{
        struct cv *cv;

        cv = kmalloc(sizeof(*cv));
        if (cv == NULL) {
                return NULL;
        }

        cv->cv_name = kstrdup(name);
        if (cv->cv_name==NULL) {
                kfree(cv);
                return NULL;
        }

        cv->cv_wchan = wchan_create(cv->cv_name);
        if (cv->cv_wchan == NULL) {
                kfree(cv->cv_name);
                kfree(cv);
                return NULL;
        }
```

```
        spinlock_init(&cv->cv_spinlock);

        return cv;
}

void
cv_destroy(struct cv *cv)
{
        KASSERT(cv != NULL);

        spinlock_cleanup(&cv->cv_spinlock);
        wchan_destroy(cv->cv_wchan);
        kfree(cv->cv_name);
        kfree(cv);
}

void
cv_wait(struct cv *cv, struct lock *lock)
{
        KASSERT(cv != NULL);
        KASSERT(lock != NULL);
        KASSERT(lock_do_i_hold(lock)); // Must hold the lock to wait on
the CV

        spinlock_acquire(&cv->cv_spinlock);

        lock_release(lock);
        wchan_sleep(cv->cv_wchan, &cv->cv_spinlock);

        spinlock_release(&cv->cv_spinlock);

        lock_acquire(lock); // Re-acquire upon waking
}

void
cv_signal(struct cv *cv, struct lock *lock)
{
        KASSERT(cv != NULL);
        KASSERT(lock != NULL);
```

```
        KASSERT(lock_do_i_hold(lock));

        spinlock_acquire(&cv->cv_spinlock);
        wchan_wakeone(cv->cv_wchan, &cv->cv_spinlock);
        spinlock_release(&cv->cv_spinlock);
}

void
cv_broadcast(struct cv *cv, struct lock *lock)
{
        KASSERT(cv != NULL);
        KASSERT(lock != NULL);
        KASSERT(lock_do_i_hold(lock));

        spinlock_acquire(&cv->cv_spinlock);
        wchan_wakeall(cv->cv_wchan, &cv->cv_spinlock);
        spinlock_release(&cv->cv_spinlock);
}
```

And output is:

OS/161 kernel [? for menu]: sy2
Starting lock test...
Lock test done.
Operation took 0.213934160 seconds
OS/161 kernel [? for menu]: sy3
Starting CV test...
Threads should print out in reverse order.
Thread 31
Thread 30
Thread 29
Thread 28
Thread 27
Thread 26
Thread 25
Thread 24
Thread 23
Thread 22
Thread 21
Thread 20
Thread 19
Thread 18

Thread 17
Thread 16
Thread 15
Thread 14
Thread 13
Thread 12
Thread 11
Thread 10
Thread 9
Thread 8
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
Thread 31
Thread 30
Thread 29
Thread 28
Thread 27
Thread 26
Thread 25
Thread 24
Thread 23
Thread 22
Thread 21
Thread 20
Thread 19
Thread 18
Thread 17
Thread 16
Thread 15
Thread 14
Thread 13
Thread 12
Thread 11
Thread 10
Thread 9
Thread 8
Thread 7
Thread 6

Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
Thread 31
Thread 30
Thread 29
Thread 28
Thread 27
Thread 26
Thread 25
Thread 24
Thread 23
Thread 22
Thread 21
Thread 20
Thread 19
Thread 18
Thread 17
Thread 16
Thread 15
Thread 14
Thread 13
Thread 12
Thread 11
Thread 10
Thread 9
Thread 8
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
Thread 31
Thread 30
Thread 29
Thread 28
Thread 27
Thread 26

Thread 25
Thread 24
Thread 23
Thread 22
Thread 21
Thread 20
Thread 19
Thread 18
Thread 17
Thread 16
Thread 15
Thread 14
Thread 13
Thread 12
Thread 11
Thread 10
Thread 9
Thread 8
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
Thread 31
Thread 30
Thread 29
Thread 28
Thread 27
Thread 26
Thread 25
Thread 24
Thread 23
Thread 22
Thread 21
Thread 20
Thread 19
Thread 18
Thread 17
Thread 16
Thread 15
Thread 14

```
Thread 13
Thread 12
Thread 11
Thread 10
Thread 9
Thread 8
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
CV test done
Operation took 3.483931960 seconds
OS/161 kernel [? for menu]:
```